॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

CASE STUDY REPORT

Submitted

By

Ms. Harshita Kanwar

Subject: Introduction to Cyber Security

Semester: III

Section: C

PRN: 25070521330

OCTOBER 2026

Index

1. Introduction

2. Objectives

3. Problem statement

4. Implementation and Procedure (Steps, Commands, Tools and Software Used)

5. Results and Output

6. Applications and Security Analysis

7. Conclusion

8. References

Case Study: Wireshark Installation and Packet Analysis

1. Introduction

Wireshark is an open-source network protocol analyzer used to capture and inspect data packets travelling through a computer network. It provides detailed information about network communication, including source and destination IP addresses, protocols, port numbers, packet lengths, and other packet-related details. This information helps users understand how devices exchange data over a network and how different protocols function during communication.

Wireshark is widely used in network troubleshooting, protocol analysis, and cybersecurity investigations. It allows users to capture live network traffic, apply display filters, and examine individual packets to identify unusual patterns or possible communication problems. It also helps in studying protocols such as TCP and TLS, making it a useful tool for understanding network behaviour and basic security concepts [1, 2].

2. Objectives

The main objectives of this case study are:

- To install Wireshark and verify its version.
- To identify and select an appropriate network interface.
- To capture and observe network packets.
- To understand the packet-list columns and protocol information.
- To apply display filters to analyse specific network traffic.
- To examine the source and destination addresses of a selected packet.
- To understand TCP ports, flags, sequence numbers, and acknowledgement numbers.
- To identify the significance and limitations of TCP retransmission indicators.
- To understand why encrypted TLS application data is not normally readable directly.

3. Problem Statement

In a computer network, data is continuously exchanged between devices and remote servers. However, the underlying packet exchange is not directly visible to users, making it difficult to understand which protocols are being used and how devices communicate.

This case study uses Wireshark to capture and analyze network traffic. The captured packets are filtered to identify TCP traffic, and an individual packet is examined to understand its source and destination IP addresses, port numbers, sequence numbers, acknowledgement numbers, and TCP flags [2, 3, 4].

The captured traffic also contains TLS 1.2 packets. This helps demonstrate that although packet headers and communication details can be examined, encrypted application data generally cannot be read directly without appropriate decryption information [6]. The objective is to understand packet analysis and protocol behaviour using Wireshark while maintaining responsible and authorized network monitoring.

4. Implementation and Procedure

This section describes the practical implementation of the Wireshark packet analysis experiment, including the software used, network interface selection, packet capture, filtering, and interpretation of packet fields.

4.1 Tools and Software Used

The following tools and software were used to perform the experiment [1, 2].

Tool/Component	Purpose
Wireshark 4.6.9	Capturing, filtering, and analysing network packets
Windows operating system	Platform on which Wireshark was executed
Wi-Fi network interface	Capturing authorised network traffic

Display filter bar	Isolating packets belonging to selected protocols
Packet details pane	Examining protocol headers and packet fields
Packet bytes pane	Viewing the hexadecimal and ASCII representation of captured bytes
PCAP/PCAPNG capture format	Storing captured packets for later analysis

4.2 Step 1: Installation and Version Verification

Wireshark was installed from its official source and opened to verify successful installation. The welcome screen displayed Wireshark version 4.6.9 and the available network interfaces [1].

4.3 Step 2: Selection of Network Interface

The available interfaces were examined, and **Wi-Fi 2** was selected because it was being used for the network activity. Selecting the correct interface helps capture the relevant traffic [2].

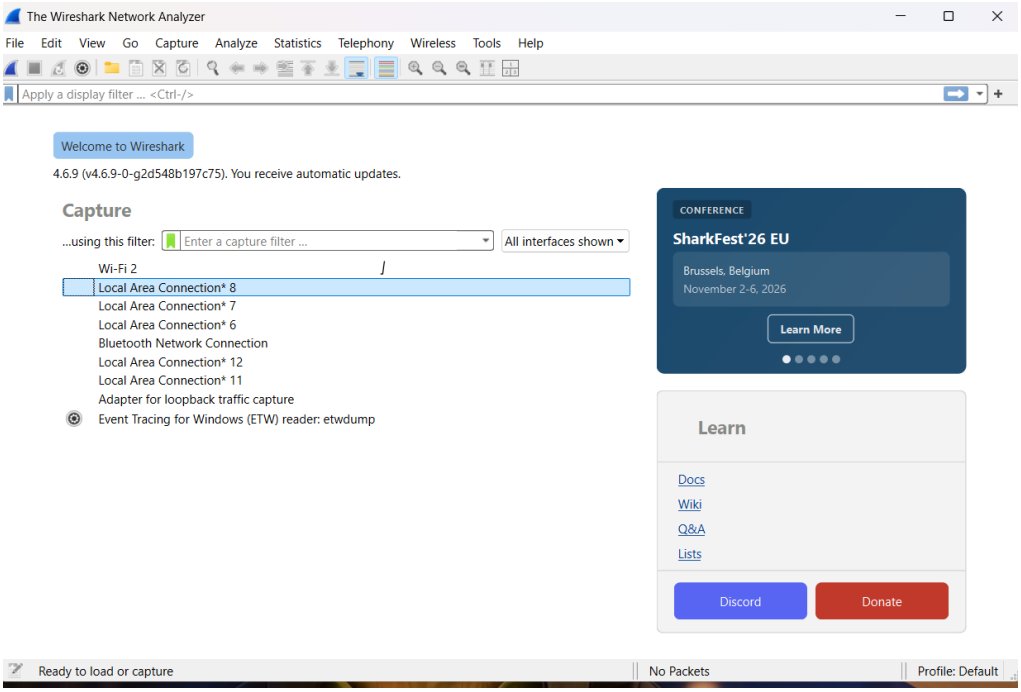


Figure 1: Selection of the Wi-Fi 2 network interface for packet capture

4.4 Step 3: Capturing Network Traffic

Packet capture was started on the selected interface. Wireshark displayed packet numbers, timestamps, source and destination addresses, protocols, and packet lengths. Approximately **15,065 packets** were captured, including TCP and TLSv1.2 traffic. The capture was then stopped and saved in PCAPNG format.

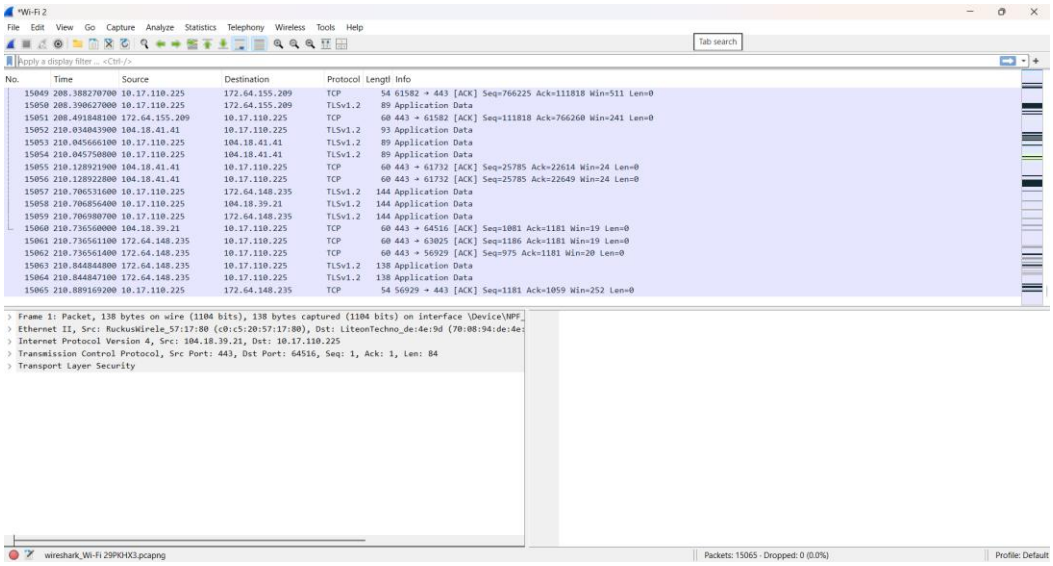


Figure 2: Captured network traffic showing TCP and TLSv1.2 packets

4.5 Step 4: Applying a Display Filter

A large packet capture can contain thousands of packets, making manual inspection time-consuming. Wireshark provides display filters to show only packets that match a specified condition.

For this experiment, the display filter tcp was entered into the filter bar. After the filter was applied, the filter field turned green, indicating that the expression was valid.

The filter displayed TCP packets and allowed unrelated protocol entries to be excluded from the visible packet list. The original capture remained available; the filter only changed which packets were displayed [2, 3].

The following display filters can be used depending on the purpose of the investigation:

Protocol	Display filter	Purpose
TCP	tcp	Displays TCP packets

DNS	dns	Displays DNS query and response packets when present
ICMP	icmp	Displays IPv4 ICMP packets, such as echo requests and replies
TLS	tls	Displays packets Wireshark identifies as TLS traffic
HTTPS-related TCP traffic	tcp.port == 443	Displays TCP packets with source or destination port 443

A display filter is different from a capture filter. A display filter is applied after packets have been captured or loaded, whereas a capture filter determines which packets are collected in the first place.

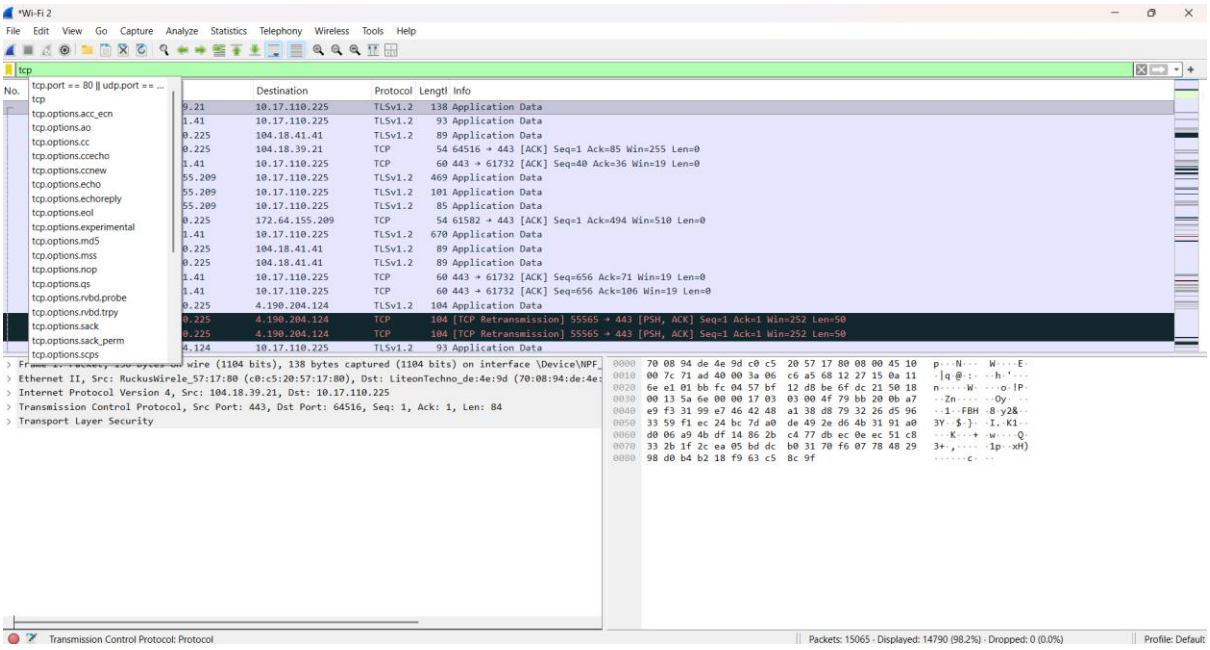


Figure 3: Application of the TCP display filter and filtered packet results

4.6 Step 5: Selecting and Examining an Individual Packet

A TCP packet was selected from the filtered list. The Packet Details pane was used to examine its protocol layers, IP addresses, port numbers, sequence and acknowledgement numbers, TCP flags, and payload length. The Packet Bytes pane showed the packet's hexadecimal and ASCII representation [2], [4].

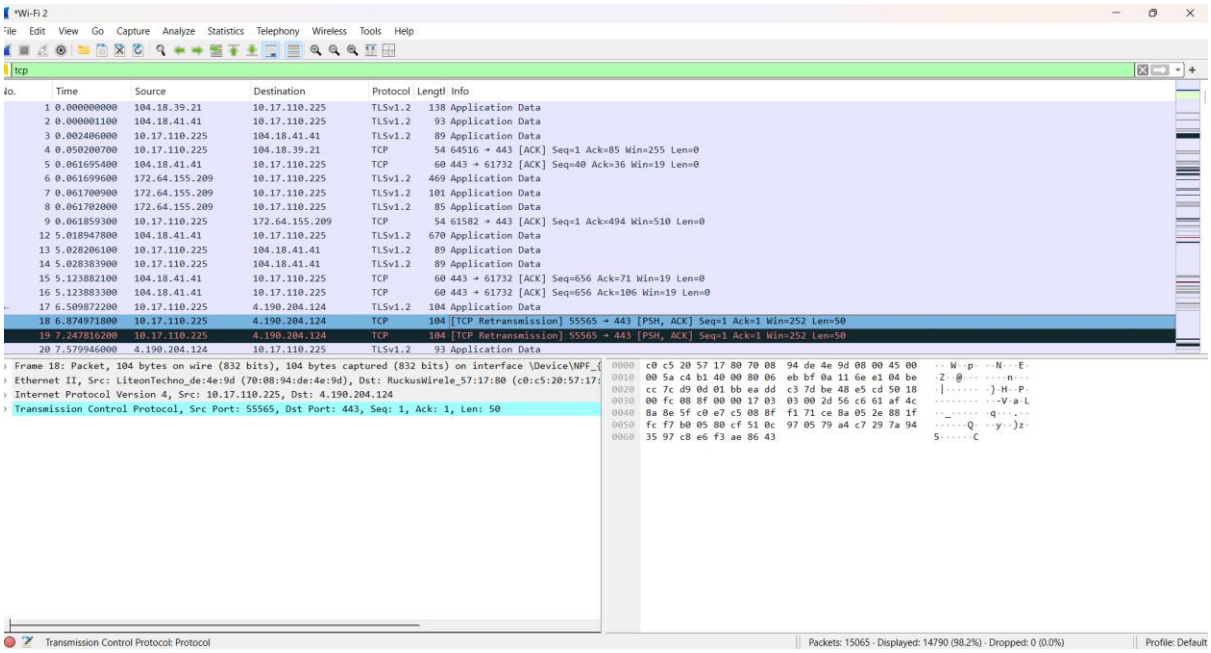


Figure 4: Detailed examination of a selected TCP packet showing source and destination ports, TCP flags, sequence numbers, and retransmission information.

4.7 Step 6: Analysing the Relevant Packet Fields

The selected packet was examined to understand the purpose of its important fields.

Field	Observed value	Interpretation
Frame number	18	Identifies the packet in the displayed capture
Source IP address	10.17.110.225	Local device sending the selected packet
Destination IP address	4.190.204.124	Remote destination receiving the packet
Protocol	TCP	Transport protocol responsible for reliable, ordered delivery
Source port	55565	Client-side port used by the connection

Destination port	443	Commonly used port for HTTPS services
Sequence number	1 (relative)	Relative TCP sequence position of the segment
Acknowledgement number	1 (relative)	Relative acknowledgement value shown by Wireshark
TCP flags	PSH, ACK	Push and acknowledgement flags are set
TCP payload length	50 bytes	Number of payload bytes carried in this TCP segment
Retransmission indication	Present	Wireshark identifies the segment as retransmitted TCP data

The capture contained TLSv1.2 traffic and TCP packets using destination port 443, commonly associated with HTTPS. A TCP retransmission indicator was also observed. This indicates a retransmission identified by Wireshark but does not establish its cause. TLS application data generally remains unreadable without appropriate decryption information [4, 6].

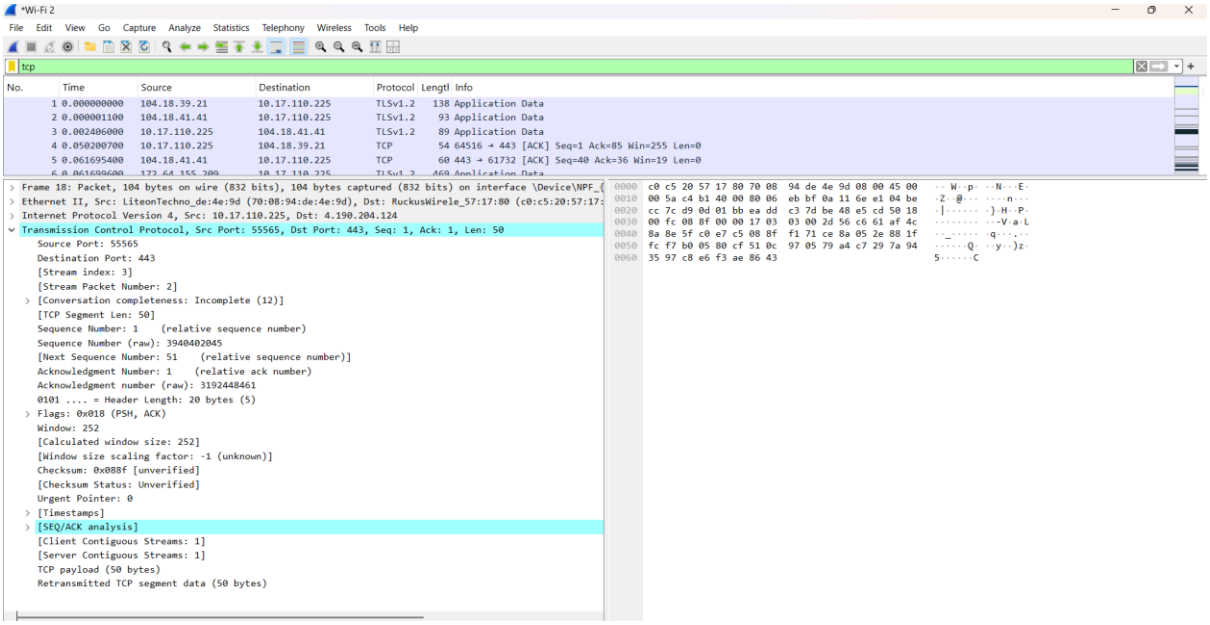


Figure 5: Interpretation of TCP header fields in Wireshark

4.8 Step 7: Understanding Encrypted Traffic

The captured packets included TLSv1.2 traffic. Wireshark displayed packet information and protocol details, but the encrypted application data was not directly readable. This demonstrates how TLS protects data exchanged over a network. Reading the protected content generally requires appropriate decryption information [6].

5. Results and Output

The practical demonstrated the basic process of capturing and analysing network traffic using Wireshark.

The following results were obtained:

1. **Installation:** Wireshark launched successfully, and version 4.6.9 was displayed.
2. **Interface selection:** Wi-Fi 2 was selected for the packet capture.
3. **Packet capture:** The screenshots showed approximately 15,065 packets, including TCP and TLSv1.2 traffic.
4. **Display filtering:** The tcp filter was accepted, showing 14,790 displayed packets out of 15,065 total packets in the illustrated capture state.

5. **Packet inspection:** Frame 18 was examined to identify its IP addresses, TCP ports, sequence and acknowledgement numbers, flags, and payload length.
6. **Retransmission observation:** Wireshark marked the selected segment as a TCP retransmission, demonstrating a feature useful in network troubleshooting.
7. **Encryption limitation:** TLSv1.2 application data was visible, but its protected content was not ordinarily readable directly from the packet capture.

Overall, the practical showed how Wireshark organises network traffic and provides detailed information for packet-level analysis.

6. Applications and Security Analysis

Wireshark has several practical applications in computer networking, cybersecurity, and system administration. It allows analysts to investigate communication patterns at the packet level rather than relying only on the information provided by applications.

6.1 Network Troubleshooting

Wireshark helps identify network problems such as TCP retransmissions, connection resets, missing acknowledgements, and delays. In this practical, a TCP retransmission was observed. Further analysis is needed to determine its actual cause.

6.2 Protocol Analysis and Learning

Wireshark helps students understand network protocols by examining TCP headers, IP addresses, DNS queries, ICMP messages, and TLS records. Its packet details pane connects theoretical concepts with real network communication.

6.3 Performance Investigation

Packet timestamps, lengths, acknowledgements, and retransmission indicators help investigate network delays and possible communication problems. However, the analysis is limited to traffic visible at the selected capture point.

6.4 Cybersecurity Monitoring

Wireshark supports authorised security investigations by helping analysts identify unusual communication patterns, unexpected connections, repeated connection

attempts, and suspicious protocol behaviour. However, it cannot automatically determine whether traffic is safe or malicious.

6.5 Limitations of Encrypted Traffic Analysis

Wireshark can display metadata such as IP addresses, ports, packet sizes, timing, and protocol information. However, TLS-encrypted application data is not generally readable directly from a packet capture. Reading it may require appropriate session keys and supporting information [2, 6].

6.6 Privacy, Authorisation, and Ethical Considerations

Packet captures may contain personal or sensitive information. Therefore, Wireshark should be used responsibly and only for authorised purposes.

- Capture traffic only from personal devices, authorised networks, or instructor-provided files.
- Do not intercept other individuals' communications without permission.
- Store packet captures securely and avoid sharing sensitive information.
- Use display filters to focus on relevant traffic.
- Do not assume encrypted or unusual traffic is automatically malicious.
- Draw conclusions only from evidence supported by the capture.

These practices help ensure responsible and ethical network analysis.

6.7 Limitations of Practical

Wireshark analyses only the traffic visible at the selected capture point and may not capture every packet across a network. TLS-encrypted application data is not normally readable without appropriate decryption information. Similarly, a TCP retransmission indicator alone cannot establish the cause of a network problem. Further investigation may be required [2,4, 6].

7. Conclusion

The case study successfully demonstrated the installation and practical use of Wireshark for network traffic analysis. Wireshark version 4.6.9 was verified, and the Wi-Fi 2 interface was used to capture network traffic containing TCP and TLSv1.2 packets.

The tcp display filter helped narrow down the packet list, while the examination of Frame 18 provided an understanding of IP addresses, TCP ports, sequence and acknowledgement numbers, flags, and payload length. The observed TCP retransmission demonstrated how Wireshark can assist in network troubleshooting.

The practical also highlighted the limitations of analysing encrypted traffic, as TLS application data is not normally readable directly from packet captures. Overall, the case study improved understanding of packet analysis, network protocols, troubleshooting, and the responsible use of network monitoring tools [2].

8. References

1. Wireshark Foundation. Wireshark: The World's Most Popular Network Protocol Analyzer. Official website. https://www.wireshark.org/

2. Wireshark Foundation. Wireshark User's Guide. Documentation covering packet capture, display filters, protocol dissection, and packet analysis. https://www.wireshark.org/docs/wsug_html_chunked/

3. Wireshark Foundation. Display Filter Reference. Reference for protocol fields and display-filter expressions. https://www.wireshark.org/docs/dfref/

4. Internet Engineering Task Force (IETF). RFC 9293: Transmission Control Protocol (TCP). https://www.rfc-editor.org/rfc/rfc9293

5. Internet Engineering Task Force (IETF). RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. https://www.rfc-editor.org/rfc/rfc8446

6. Internet Engineering Task Force (IETF). RFC 5246: The Transport Layer Security (TLS) Protocol Version 1.2. https://www.rfc-editor.org/rfc/rfc5246

Case Study 2: Firewall Configuration and Rule Evaluation

1. Introduction

A firewall is a security mechanism that monitors and controls incoming and outgoing network traffic based on predefined security rules. It helps protect a computer from unauthorised access and unwanted network connections.

Windows Defender Firewall is a built-in security feature of the Windows operating system. It uses inbound and outbound rules to determine which network connections should be allowed or blocked. These rules may depend on the application, network profile, protocol, port, and other connection conditions.

Firewall rule evaluation is important because incorrectly configured rules can expose a system to unnecessary security risks. This case study focuses on inspecting Windows Defender Firewall, examining existing rules, and understanding how firewall profiles, protocols, ports, and actions influence network security [1].

2. Objectives

The main objectives of this case study are:

1. To identify Windows Defender Firewall and inspect its current status.
2. To locate and examine inbound and outbound firewall rules.
3. To inspect an existing firewall rule and understand its properties.
4. To identify the protocol and port associated with the selected rule.
5. To examine the configuration of Domain, Private, and Public firewall profiles.
6. To understand how firewall rules help prevent unauthorised network access.
7. To analyse the security risks associated with broad or unnecessary allow rules.

3. Problem Statement

Computers connected to networks may receive unwanted connection requests or communicate with external systems through applications and services. Without appropriate firewall controls, unnecessary network traffic may increase the risk of unauthorised access.

The problem addressed in this case study is to understand how Windows Defender Firewall manages network traffic through its rules and profiles. The existing firewall configuration must be inspected to identify enabled rules, their direction, permitted actions, and associated protocol and port information [2, 3, 4].

The study also examines why narrowly scoped firewall rules are important for reducing security risks. The configuration is inspected without disabling the firewall globally or making unauthorised changes to existing rules.

4. Implementation and Procedure

4.1 Tools and Software Used

- Operating System: Microsoft Windows
- Firewall Utility: Windows Defender Firewall
- Advanced Management Tool: Windows Defender Firewall with Advanced Security (wf.msc) [1]
- Command-Line Tool: Windows PowerShell

4.2 Step 1: Checking Firewall Status

Windows Defender Firewall was opened through the Control Panel. The displayed network profile was inspected to determine whether the firewall was enabled.

The firewall status showed that protection was enabled for the active Public network profile [1].

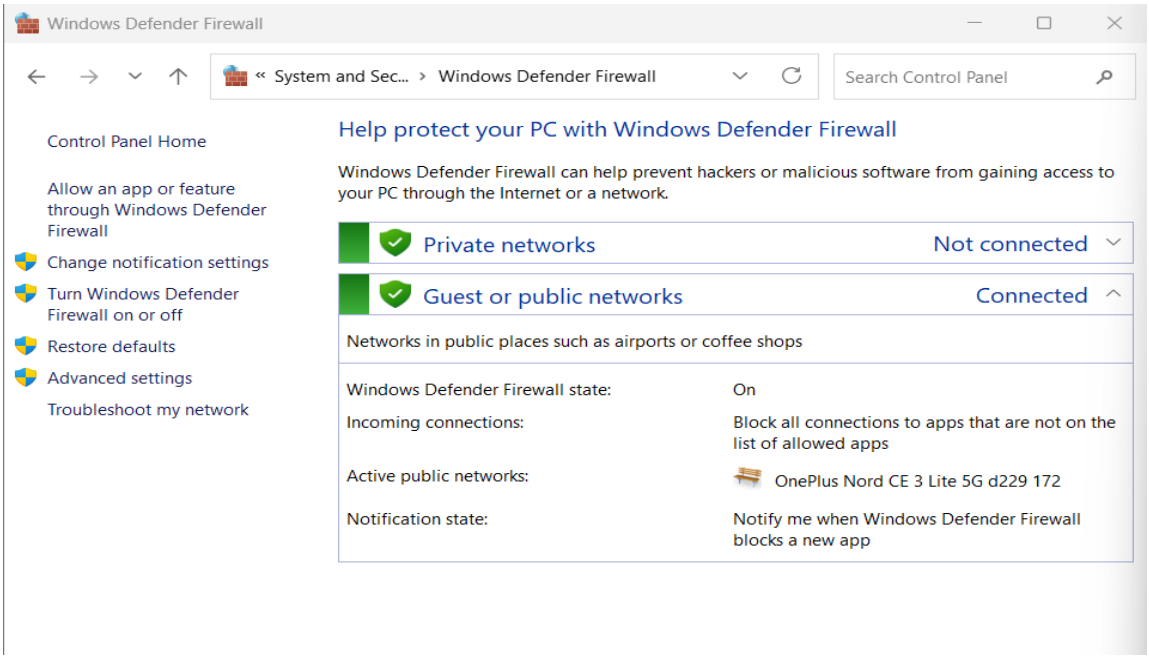


Figure 1: Windows Defender Firewall Status Showing the Firewall Enabled for the Public Network profile

4.3 Step 2: Inspecting Inbound Rules and outbound rules

The Run dialog was opened using Windows + R. The command wf.msc was entered to launch Windows Defender Firewall with Advanced Security.

The Inbound Rules section was selected to inspect the available rules controlling incoming network traffic. These rules determine which incoming connections are permitted or blocked when their conditions match [1].

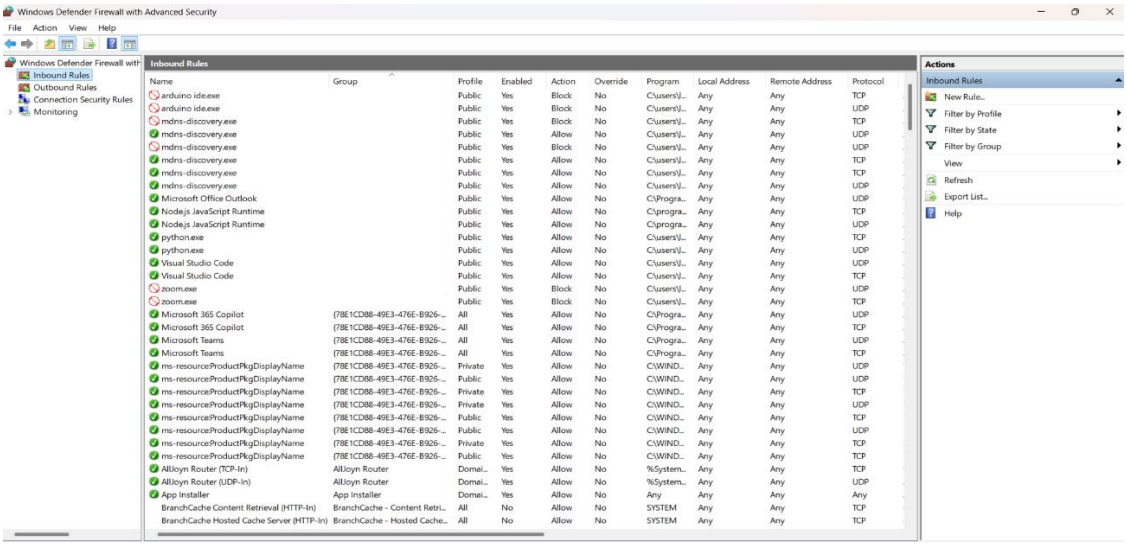


Figure 2: Inbound Firewall Rules Displayed in Windows Defender Firewall with Advanced Security5

The Outbound Rules section was selected in the same management window. The available rules were examined to understand how Windows controls outgoing connections initiated by applications or services on the computer [1].

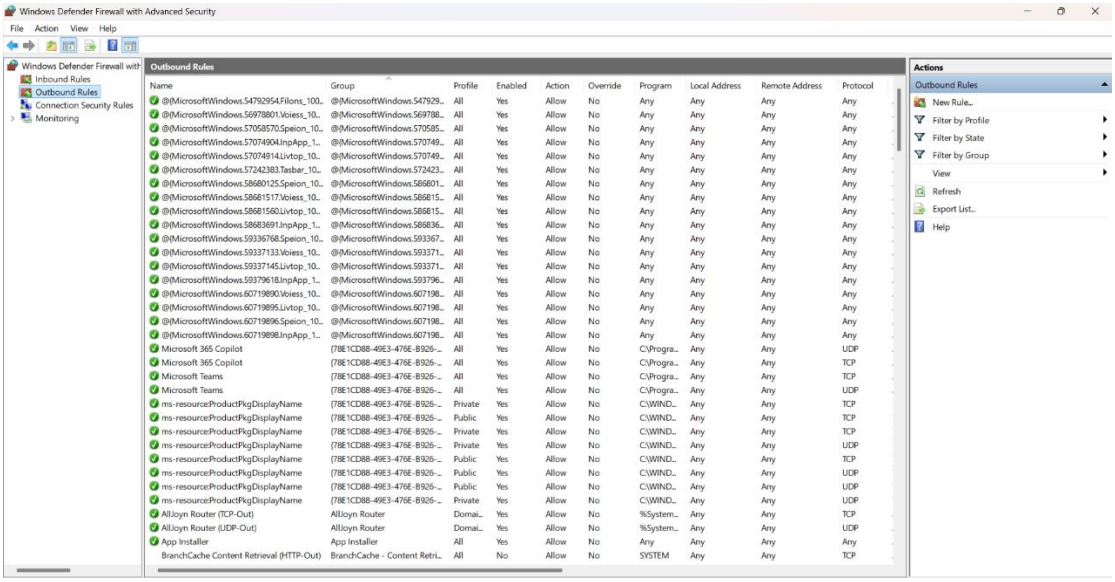


Figure 3: Outbound Firewall Rules Displayed in Windows Defender Firewall with Advanced Security

4.4 Step 3: Inspecting Firewall Profiles

The configuration of the Domain, Private, and Public firewall profiles was inspected using the following command:

```
Get-NetFirewallProfile |
Format-Table Name, Enabled, DefaultInboundAction, DefaultOutboundAction -AutoSize
```

The output showed that all three profiles were enabled. The default inbound and outbound action fields appeared as NotConfigured.

This indicates that the command did not show an explicitly configured default action in those fields. The effective policy should not be inferred from this output alone because Windows defaults or centrally managed policies may also affect the resulting configuration [3].

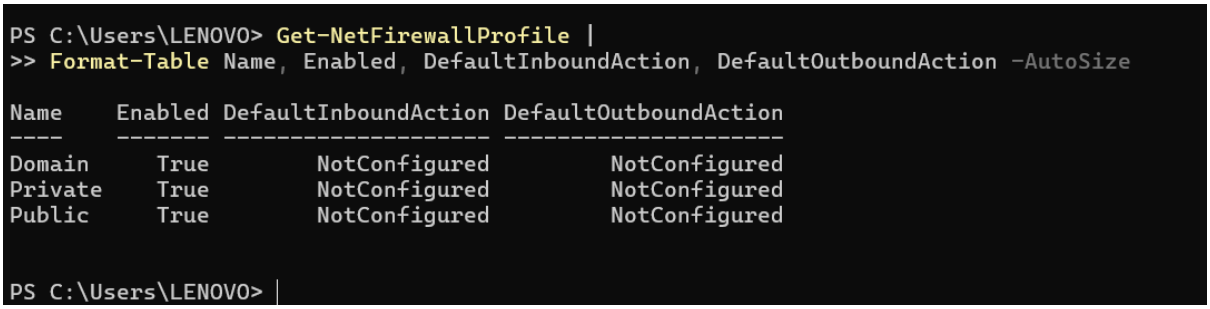


Figure 6: PowerShell Output Showing the Status and Default Action Fields of Windows Firewall Profiles

4.5 Step 4: Configuring a Restricted Test Rule

To evaluate how firewall policies enforce access control, a custom inbound block rule named **CaseStudy-TestRule-Disabled** was configured for TCP port 8080 on the Public network profile using PowerShell [2]:

New-NetFirewallRule -DisplayName "CaseStudy-TestRule-Disabled" -Direction Inbound -Protocol TCP -LocalPort 8080 -Profile Public -Action Block

Rule Property	Setting
Display Name	CaseStudy-TestRule-Disabled
Direction	Inbound
Protocol	TCP
Local Port	8080
Network Profile	Public
Action	Block

```
PS C:\Users\LENOVO> New-NetFirewallRule -DisplayName "CaseStudy-TestRule-Disabled" -Direction
Inbound -Protocol TCP -LocalPort 8080 -Profile Public -Action Block


Name                            : {5d61c3b5-6810-44fd-b3ca-09673577a169}
DisplayName                     : CaseStudy-TestRule-Disabled
Description                     :
DisplayGroup                    :
Group                           :
Enabled                         : True
Profile                         : Public
Platform                        : {}
Direction                       : Inbound
Action                          : Block
EdgeTraversalPolicy             : Block
LooseSourceMapping              : False
LocalOnlyMapping                : False
Owner                           :
PrimaryStatus                   : OK
Status                          : The rule was parsed successfully from the
                                  store. (65536)
EnforcementStatus               : NotApplicable
PolicyStoreSource               : PersistentStore
PolicyStoreSourceType           : Local
RemoteDynamicKeywordAddresses   : {}
PolicyAppId                     :
PackageFamilyName               :
```

Figure 5: Creation of the custom inbound block rule CaseStudy-TestRule-Disabled in PowerShell

4.6 Step 5: Verifying Rule Properties and Port Filters

After creating the rule, its configuration was verified using PowerShell commands to confirm that the port filter, profile, and block action were applied correctly [2, 4]:

Get-NetFirewallRule -DisplayName "CaseStudy-TestRule-Disabled" | Format-List DisplayName, Enabled, Direction, Action, Profile

Get-NetFirewallRule -DisplayName "CaseStudy-TestRule-Disabled" | Get-NetFirewallPortFilter | Format-List Protocol, LocalPort, RemotePort

```
PS C:\Users\LENOVO> Get-NetFirewallRule -DisplayName "CaseStudy-TestRule-Disabled" | Format-List DisplayName, Enabled, Direction, Action, Profile


DisplayName : casestudy-testrule-disabled
Enabled     : True
Direction   : Inbound
Action      : Block
Profile     : Public

DisplayName : CaseStudy-TestRule-Disabled
Enabled     : True
Direction   : Inbound
Action      : Block
Profile     : Public
```

Figure 5.1: Verification of the firewall rule configuration showing DisplayName, Enabled status (True), Direction (Inbound), Action (Block), and Profile (Public).



Figure 5.2: Verification of the firewall rule's port filter showing Protocol (TCP), LocalPort (8080), and RemotePort (Any).

4.7 Step 6: Evaluating Network Connectivity

To test the enforcement of the configured firewall rule, a TCP connection attempt was made targeting port 8080 [5]:

Test-NetConnection -ComputerName Localhost -Port 8080

The command returned **TcpTestSucceeded: False**, confirming that incoming TCP connections to port 8080 were successfully blocked by the rule.

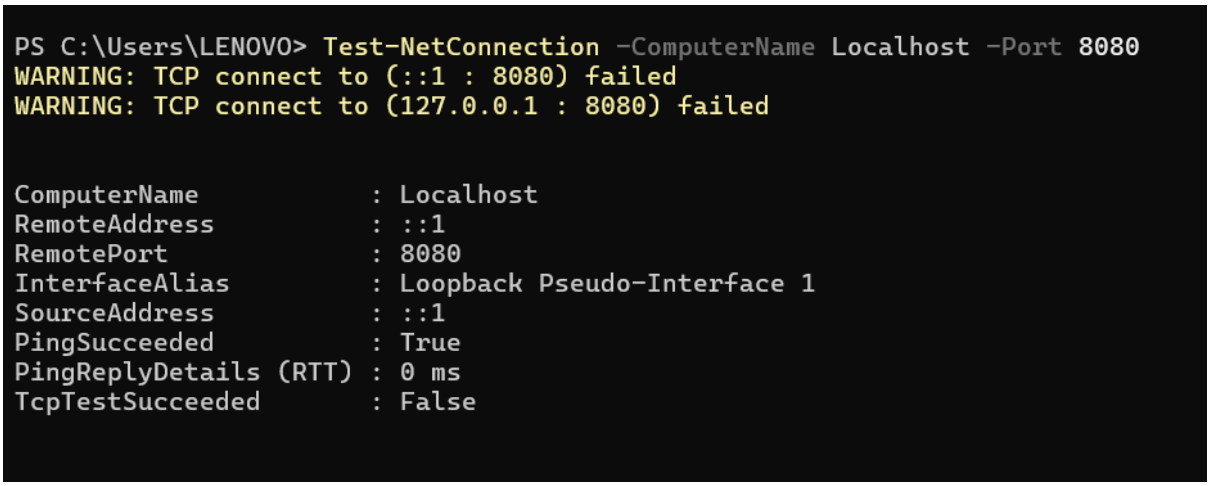


Figure 7: Connectivity test output showing TcpTestSucceeded: False

4.8 Step 7: Reversing the Test Configuration (System Baseline Restoration)

After completing the testing and verification, the temporary test rule was removed using PowerShell to prevent configuration drift and restore the system to its baseline security posture [2, 5]:

Remove-NetFirewallRule -DisplayName "CaseStudy-TestRule-Disabled"

Get-NetFirewallRule -DisplayName "CaseStudy-TestRule-Disabled"

Querying the rule returned an ObjectNotFound error, confirming that **CaseStudy-TestRule-Disabled** was successfully purged from the system.

```
PS C:\Users\LENOVO> Remove-NetFirewallRule -DisplayName "CaseStudy-TestRule-
Disabled"
PS C:\Users\LENOVO> Get-NetFirewallRule -DisplayName "CaseStudy-TestRule-Dis
abled"
Get-NetFirewallRule : No MSFT_NetFirewallRule objects found with property
'DisplayName' equal to 'CaseStudy-TestRule-Disabled'.  Verify the value of
the property and retry.
At line:1 char:1
+ Get-NetFirewallRule -DisplayName "CaseStudy-TestRule-Disabled"
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (CaseStudy-TestRule-Disabled
   :String) [Get-NetFirewallRule], CimJobException
    + FullyQualifiedErrorId : CmdletizationQuery_NotFound_DisplayName,Get-
   NetFirewallRule
```

Figure 8: Removal of the test rule and verification of system baseline cleanup

5. Results and Output

The inspection of Windows Defender Firewall was completed successfully using the graphical interface and Windows PowerShell.

The following results were obtained:

S. No.	Parameter / Observation	Recorded Result
1	**Firewall Product**	Windows Defender Firewall
2	**Firewall Status**	Enabled
3	**Active Profiles**	Domain, Private, Public (All Profiles Enabled)
4	**Configured Test Rule**	CaseStudy-TestRule-Disabled
5	**Direction and Action**	Inbound, Block
6	**Protocol and Port**	TCP, Local Port 8080
7	**Rule Status**	Enabled (True)

8	**Verification Methods**	PowerShell Rule Inspection & Port Filter Check
9	**Connectivity Test**	Test-NetConnection -ComputerName Localhost -Port 8080
10	**Observed Outcome**	Connection failed (TcpTestSucceeded: False)
11	**Post-Test Cleanup**	Rule removed via Remove-NetFirewallRule (ObjectNotFound verified)

The results demonstrate how PowerShell and advanced management tools can be used to configure, verify, and test firewall rule properties. The custom rule **CaseStudy-TestRule-Disabled** successfully intercepted and blocked incoming TCP connection attempts on port 8080 under the Public profile, as confirmed by the connectivity test outcome (TcpTestSucceeded: False). Furthermore, system cleanup successfully restored the workstation's original security baseline.

6. Applications and Security Analysis

Applications

Windows Defender Firewall and firewall rule evaluation are useful in the following areas:

1. **Personal computer security**: Helps restrict unwanted incoming network connections.
2. **Application control**: Allows administrators to inspect and manage network access for individual applications.
3. **Enterprise network security**: Supports the application of network access policies across managed computers.
4. **Public network protection**: Helps reduce exposure when using less-trusted networks.
5. **Network troubleshooting**: Helps administrators examine rules, protocols, ports, and profiles when investigating connection problems [1,4].

Security Analysis

Firewall rules help reduce risk by controlling which network traffic is permitted. However, the effectiveness of a rule depends on its configuration and the traffic to which it applies.

- **Inbound traffic**: Unnecessary inbound access may expose applications or services to unwanted connection attempts.
- **Outbound traffic**: Outbound rules can help restrict unauthorised or unnecessary communication initiated by applications.
- **Broad allow rules**: Rules that permit more traffic than required may increase the system's attack surface.
- **Network profiles**: A rule permitted on a Public profile should be reviewed carefully because public networks are generally less trusted.
- **Least privilege**: Rules should allow only the necessary applications, protocols, ports, sources, and network profiles [1, 2, 4].
- **Regular inspection**: Existing rules should be reviewed periodically to identify unnecessary or outdated permissions.

In this case study, a narrowly scoped inbound block rule (CaseStudy-TestRule-Disabled) was successfully configured, verified, tested for connectivity, and safely removed. The practical demonstrates how precise rule scoping helps enforce the principle of least privilege and protect local services from unwanted network exposure.

7. Conclusion

This case study demonstrated the inspection, evaluation, configuration, and cleanup of Windows Defender Firewall using Windows PowerShell and the Advanced Security console. By configuring a custom inbound block rule (**CaseStudy-TestRule-Disabled**), verifying its port filter settings, evaluating its effect via Test-NetConnection, and safely purging the rule afterward, the practical illustrated how firewall rules effectively regulate network access [2, 4, 5]. These practices reinforce the principle of

least privilege, reduce unnecessary network exposure, and maintain robust system integrity.The practical helped explain how firewall rules control incoming and outgoing network traffic and how their configuration affects system security. It also highlighted the importance of avoiding unnecessarily broad allow rules and following the principle of least privilege. Proper inspection and maintenance of firewall rules contribute to reducing unnecessary network exposure and improving computer security.

8. References

1. Microsoft Learn, Windows Defender Firewall with Advanced Security. https://learn.microsoft.com/en-us/windows/security/operating-system-security/network-security/windows-firewall/
2. Microsoft Learn, Get-NetFirewallRule PowerShell Reference. https://learn.microsoft.com/en-us/powershell/module/netsecurity/get-netfirewallrule
3. Microsoft Learn, Get-NetFirewallProfile PowerShell Reference. https://learn.microsoft.com/en-us/powershell/module/netsecurity/get-netfirewallprofile
4. Microsoft Learn, Get-NetFirewallPortFilter PowerShell Reference. https://learn.microsoft.com/en-us/powershell/module/netsecurity/get-netfirewallportfilter
5. Practical observations and command outputs recorded during the case study.